5-STAGE PIPELINED PROCESSOR

Abdo Moamen Eltaweel

Table of content

1. Introduction **3**
1.1 Project Overview 3
1.2 Design Objectives 3
2. Overall Processor Architecture **4**
2.1 High-Level Processor Architecture 4
2.2 Pipeline Organization 5
2.3 Data and Control Flow Summary 5
3. Core Functional Blocks Design **6**
3.1 Program Counter (PC) 6
3.2 Register File (RF) 7
3.3 Arithmetic Logic Unit (ALU) 9
3.4 Condition Code Register (CCR) 10
3.5 Control Unit 12
3.6 Jump Unit 14
3.7 Forwarding Unit 17
3.8 Hazard Detection Unit 20
3.9 Von Neumann Memory Unit 22
3.10 Pipeline Registers 24
4. Synthesis And Implementation **27**
4.1 Synthesis of the Processor Design 27
4.2 Elaboration 29
4.3 Implementation 30
5. Test Cases and Verification **32**
5.1 Arithmetic and Logical Instructions 32
5.2 Loop Instructions 33
5.3 Memory Access Instructions 33
5.4 Control Flow Instructions 34
5.5 Advanced test cases 35
6. Conclusion **40**

1. Introduction

1.1 Project Overview

This project presents the design and implementation of an 8-bit pipelined processor developed as part of the **Advanced Microprocessors** course. The processor is designed to support a custom instruction set that includes arithmetic, logical, memory access, control-flow, and interrupt-handling instructions. All instructions have been fully implemented and verified.

The processor follows a modular design approach, where each functional unit is implemented as an independent block with a clearly defined role. The main components of the processor include the Program Counter (PC), Von Neumann memory model (Memory), Register File (RF), Control Unit (CU), Arithmetic Logic Unit (ALU), Condition Code Register (CCR), Jump Unit (JU), Forward Unit (FU), Hazard unit (HU), and pipeline registers Fetch To Decode (F/D), Decode To Execute (D/E), Execute To Memory (E/M) and Memory To Write Back (M/WB) to separating the different pipeline stages.

To improve performance, the processor employs a pipelined architecture consisting of five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipelining allows multiple instructions to be processed concurrently, increasing instruction throughput while maintaining a relatively simple hardware structure.

Special attention was given to control-flow instructions, stack-based operations, and interrupt handling. As a result, dedicated mechanisms such as a separate Jump Unit and an enhanced Condition Code Register were introduced to ensure correct execution flow and reliable interrupt processing.

1.2 Design Objectives

The primary objective of this project was to design a functional and extensible processor architecture that balances simplicity, clarity, and performance. The main design goals can be summarized as follows:

- **Modularity:** Each functional block was designed independently to simplify debugging, testing, and future extensions.
- **Clear separation of responsibilities:** Control-flow decisions were isolated from instruction decoding by introducing a dedicated Jump Unit.
- **Support for stack operations:** The architecture supports stack-based instructions such as PUSH, POP, CALL, and RET through a dedicated stack pointer mechanism.
- **Interrupt handling capability:** The processor includes a structured interrupt handling mechanism with flag preservation and restoration.
- **Pipeline correctness:** Proper handling of control hazards through pipeline flushing and control signal management.
- **Scalability:** The design allows for future expansion, including additional instructions, hazard detection units, and performance optimizations.

These objectives guided all architectural and implementation decisions throughout the project and ensured that the final design is both robust and adaptable.

2. Overall Processor Architecture

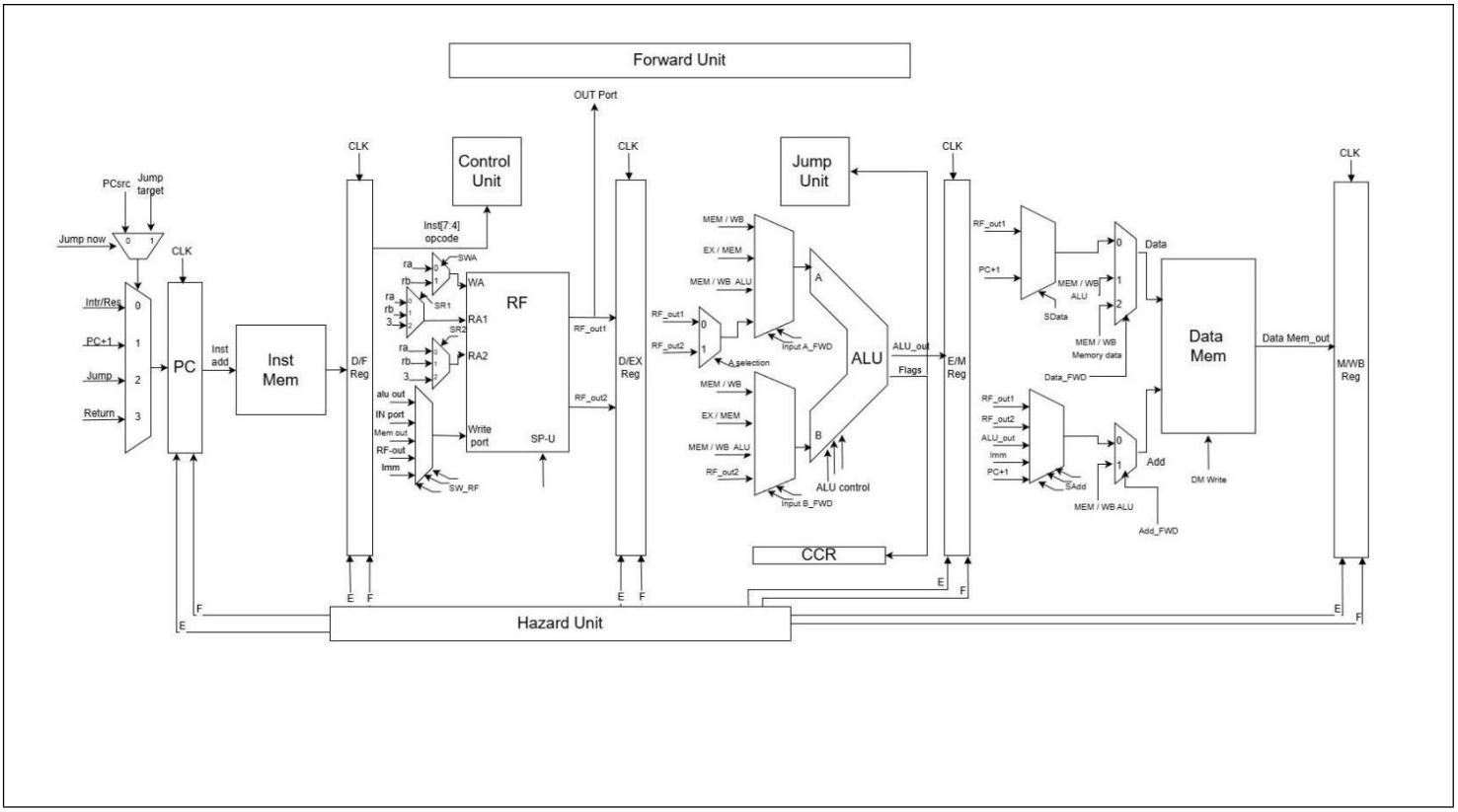
2.1 High-Level Processor Architecture

The designed processor follows an 8-bit pipelined architecture based on a **Von Neumann memory model**, where a single memory is used for both instruction storage and data storage. The processor is composed of several modular functional blocks that cooperate to execute instructions efficiently while supporting control-flow operations and interrupt handling.

At a high level, the processor consists of the following main components:

- Program Counter (PC)
- Unified Memory (Instruction and Data Memory)
- Control Unit
- Register File (RF)
- Arithmetic Logic Unit (ALU)
- Condition Code Register (CCR)
- Jump Unit
- Hazard Unit
- Forwarding Unit
- Pipeline Registers (IF/ID, ID/EX, EX/MEM, MEM/WB)
- Supporting multiplexers for data and control selection

Each block is designed to perform a specific role, and all blocks are connected through well-defined data paths and control signals, ensuring a clear separation of responsibilities across the processor.



2.2 Pipeline Organization

The processor is organized as a **five-stage pipeline**, allowing multiple instructions to be processed concurrently. The pipeline stages are:

1. **Instruction Fetch (IF)**
   The Program Counter (PC) supplies the address of the next instruction to memory. The fetched instruction and the incremented PC value are stored in the IF/ID pipeline register.
2. **Instruction Decode (ID)**
   The instruction is decoded by the Control Unit, which generates the required control signals. The Register File is accessed to read source operands. Hazard mitigation is supported through forwarding and pipeline control mechanisms.
3. **Execute (EX)**
   The ALU performs arithmetic and logical operations. The Jump Unit evaluates control-flow instructions such as conditional jumps, loops, calls, returns, and interrupts. Decisions made in this stage determine whether the pipeline should continue sequential execution or redirect the PC.
4. **Memory Access (MEM)**
   Data memory operations are performed during this stage. For stack-related instructions, the stack pointer is updated and memory is accessed accordingly.
5. **Write Back (WB)**
   Results from the ALU or memory are written back to the Register File. Special handling is provided for stack pointer updates and control-flow instructions.

.

2.3 Data and Control Flow Summary

During normal execution, instructions flow sequentially through the pipeline stages, with data being transferred between blocks via pipeline registers. Control signals generated in the decode stage propagate alongside the data to ensure synchronized execution. When a control-flow instruction or interrupt occurs, the Jump Unit redirects the PC and flushes invalid instructions from the pipeline to maintain correct execution.

3. Core Functional Blocks Design

3.1 Program Counter (PC)

The Program Counter (PC) is responsible for holding and updating the address of the next instruction to be fetched from memory. In this design, the PC is implemented as a sequential block with additional control logic to support pipelining, control-flow instructions, stalling, flushing, and interrupt handling.

➢ PC Update Mechanism

The PC is updated on the rising edge of the clock. The next PC value is first computed combinationally and then registered synchronously. This separation ensures correct timing behavior in a pipelined environment.

The module internally maintains two PC-related signals:

- **PC_next**: the registered PC value used as the current instruction address
- **PC_current**: the combinationally computed next value to be loaded into PC_next

➢ PC Source Selection

To support multiple control-flow scenarios, the PC uses a **selection mechanism** controlled by a 3-bit selector. Two different sources can determine the PC selection:

- **Normal execution control (**PCSrc)
- **Immediate jump control (**jump_target_Scr)

A control signal (jump_now) selects which source is active. This allows urgent control-flow changes, such as jumps or interrupts, to override normal PC update behavior without interfering with pipeline operation.

➢ Supported PC Update Sources

Depending on the selected control signal, the PC can be updated from one of the following sources:

- **Reset or interrupt target address**
  Used during reset or interrupt handling.
- **Sequential execution (PC + 1)**
  Normal instruction flow.
- **Jump target address**
  Used for jump, loop, and call instructions.
- **Return address**
  Used for return and return-from-interrupt instructions, typically retrieved from the stack.

➢ Pipeline Control Support

The PC module includes explicit support for pipeline control:

- **PC Enable (**PC_E**)**
  When deasserted, the PC holds its current value, effectively stalling instruction fetch.
- **PC Flush (**PC_F**)**
  When asserted, the PC is forced to a known safe value, flushing incorrect instructions from the pipeline. This signal has the highest priority.

➢ PC Increment Output

In addition to the main PC value, the module provides a **PC+1 output**, which is used by other blocks (such as subroutine call handling) without requiring recomputation. This improves clarity and modularity in the datapath design.

3.2 Register File (RF)

The Register File is responsible for storing and providing operands for instruction execution, as well as receiving results during the write-back stage. In this design, the Register File is tailored to support **general-purpose operations, stack-based instructions, and pipelined execution**, making it one of the most critical components of the processor.

➢ Register Organization

The Register File consists of four 8-bit registers, labeled R0 through R3.

- Registers R0–R2 are general-purpose registers
- Register R3 is reserved as the Stack Pointer (SP)

During reset, the general-purpose registers are cleared, while the stack pointer is initialized to the top of memory. This initialization simplifies stack management and ensures correct behavior for stack-related instructions such as PUSH, POP, CALL, and RET.

➢ Read Port Structure

The Register File provides **two independent read ports**, allowing two operands to be accessed simultaneously during the instruction decode stage. Each read port can select its source independently from:

- Register specified by Ra
- Register specified by Rb
- Stack Pointer (R3)

Selection of the read addresses is controlled through multiplexers driven by control signals. This flexible addressing scheme allows stack operations and arithmetic instructions to coexist without additional hardware complexity.

➢ Write Port Structure

To support the processor's instruction set and pipelined execution, the Register File supports **multiple write-back sources** and **special handling for the stack pointer**.

Write Data Selection

The data written to the Register File can originate from one of several sources:

- ALU resul
- Input port
- Data memory output
- Forwarded register value

A write-data multiplexer selects the appropriate source based on control signals generated by the Control Unit.

➢ Destination Register Selection

The destination register for write-back is selected dynamically:

- Either Ra or Rb, depending on the instruction type
- Selection is performed during the write-back stage to ensure correct pipelined behavior

➢ Stack Pointer Update Mechanism

A key design feature of this Register File is the **independent stack pointer write path**. Stack-related instructions often require simultaneous updates:

- Writing data to a general-purpose register
- Updating the stack pointer value

To support this, the Register File includes a **dedicated write enable and data path for the stack pointer (R3)**. When both a general write and a stack pointer update occur in the same cycle, the stack pointer update is given higher priority. This ensures correct stack behavior without requiring additional pipeline stages or complex arbitration logic.

➢ Write Priority and Conflict Resolution

Write conflicts are resolved deterministically:

- General-purpose register writes are blocked from targeting R3
- Stack pointer updates are handled separately and have higher priority

This approach prevents accidental overwriting of the stack pointer while maintaining compatibility with the existing write-back mechanism.

3.3 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the core computational block of the processor. It performs all arithmetic, logical, and bit-manipulation operations required by the instruction set. In addition to generating computational results, the ALU updates the processor condition flags and supports full data forwarding to enable efficient pipelined execution.

This ALU is designed to operate within a **five-stage pipelined architecture** and is tightly integrated with the **Forwarding Unit**, **Control Unit**, and **Condition Code Register (CCR)**.

➢ Operand Selection and Forwarding Support

To resolve data hazards without stalling the pipeline, the ALU supports **multiple operand sources** for both input operands A and B.

- **Input A** may be selected from:
    - Register File outputs
    - EX/MEM stage ALU result
    - MEM/WB stage ALU result
    - MEM/WB stage memory load data
- **Input B** may be selected from:
    - Decode-stage operand (register or immediate)
    - MEM/WB stage ALU result
    - MEM/WB stage memory load data

Selection is controlled by two 2-bit signals:

- aluInputASelect
- aluInputBSelect

These signals are generated by the **Forwarding Unit** and allow the ALU to receive the most recent value of an operand even if it has not yet been written back to the register file. This design significantly reduces pipeline stalls due to data dependencies.

➢ Operation Control

The specific ALU operation is determined by a 4-bit control signal (alu_op) generated by the Control Unit. Supported operations include:

- **Arithmetic**: ADD, SUB, INC, DEC, NEG
- **Logical**: AND, OR, NOT
- **Shift / Rotate**: Rotate Left through Carry (RLC), Rotate Right through Carry (RRC)
- **Flag Operations**: SETC, CLRC
- **NOP**: No operation

Each operation is executed combinationally, producing an 8-bit result.

➢ Condition Flags Handling

The ALU generates four condition flags:

- **Z (Zero)**: Set when the result equals zero
- **N (Negative)**: Reflects the MSB of the result
- **C (Carry)**: Indicates carry or borrow in arithmetic and rotate operations
- **V (Overflow)**: Indicates signed arithmetic overflo

Flag behavior is carefully designed:

- Flags are **updated only by instructions that affect them**
- For instructions that do not modify certain flags, the previous flag values are preserved by using the incoming flag values (FLAGS_in)
- Rotate-through-carry instructions correctly integrate the carry flag into the data path

The ALU outputs the updated flags as a 4-bit vector, which is later written into the CCR when enabled by the Control Unit.

➢ Rotate-Through-Carry Operations

Rotate instructions incorporate the carry flag as part of the rotation mechanism:

- In rotate-left-through-carry, the previous carry flag is shifted into the least significant bit.
- In rotate-right-through-carry, the previous carry flag is shifted into the most significant bit.

These operations update both the carry flag and the standard condition flags, enabling efficient bit manipulation and multi-precision arithmetic.

➢ Pipeline Compatibility

The ALU is fully combinational and produces its output within a single cycle, making it suitable for the **Execute (EX) stage** of the pipeline. Its integration with forwarding paths ensures correct execution of dependent instructions without unnecessary stalls.

3.4 Condition Code Register (CCR)

The Condition Code Register (CCR) stores the processor status flags generated by the ALU and makes them available to other units such as the control unit and jump unit. These flags are essential for implementing conditional execution, loops, and interrupt handling.

➢ CCR Structure

The CCR is implemented as an 8-bit register divided into two logical parts:

- **Bits [7:4] – Active Flags**
  These bits hold the current processor status flags:
  - **V**: Overflow flag
  - **C**: Carry flag
  - **N**: Negative flag
  - **Z**: Zero flag
- **Bits [3:0] – Saved Flags**
  These bits are used to temporarily store the active flags when an interrupt occurs

Only the active flags are exposed outside the CCR and used by the control and jump logic.

➢ Normal Operation (ALU Flag Update)

During normal instruction execution, the CCR updates its active flags directly from the ALU:

- When the write enable signal is asserted, the flags generated by the ALU replace the current active flags.
- This allows arithmetic, logical, rotate, and loop instructions to correctly affect the processor state.
- This update is synchronized with the clock to ensure correct behavior in the pipelined architecture.

➢ Interrupt Flag Saving

When an interrupt occurs, the current processor state must be preserved:

- Upon assertion of the interrupt save signal, the active flags are copied into the saved flag field.
- This ensures that the exact processor status is preserved before entering the interrupt service routine.

No other flag updates are allowed during this operation, giving interrupt handling higher priority.

➢ Interrupt Flag Restoration

When returning from an interrupt:

- The saved flags are copied back into the active flag field.
- This restores the processor state exactly as it was before the interrupt occurred.

This mechanism guarantees transparent interrupt handling without corrupting the execution state of the interrupted program.

3.5 Control Unit

The Control Unit is the central decision-making block of the processor. It decodes the fetched instruction and generates all necessary control signals required to drive the datapath across the pipeline stages. Its responsibilities include controlling ALU operations, register file access, memory operations, program counter updates, condition flag updates, stack manipulation, interrupt handling, and providing metadata for hazard detection and forwarding.

This Control Unit is fully designed for a **pipelined processor** and cooperates closely with the **Forwarding Unit**, **Hazard Detection Unit**, **Jump Unit**, **ALU**, **CCR**, **Register File**, and **Memory**.

➢ Instruction Decoding

The Control Unit decodes the instruction based on:

- op_code: Determines the instruction type
- ra and rb: Select source and destination registers
- Processor flags (Z, N, V, C) for conditional operations
- Interrupt signal, which has the highest priority

Each instruction class produces a specific combination of control signals that flow through the pipeline registers.

➢ ALU Control and Operand Selection

The Control Unit generates:

- **ALUControl**: Selects the ALU operation (add, subtract, logic, rotate, increment, decrement, etc.)
- **ALUSrc**: Controls whether the ALU operand comes from a register, stack pointer, or immediate value

Arithmetic, logical, shift, stack, and loop operations are all mapped to their corresponding ALU functions.

➢ Register File Control

The Control Unit manages all Register File interactions:

- **RFwrite** enables or disables register writes
- **writeAddressSel** selects between ra and rb as the destination register
- **writePortSel** selects the write-back data source (ALU result, memory output, input port, etc.)
- **readAddressASel / readAddressBSel** select which registers are read, including support for the stack pointer

Special care is taken to handle instructions that simultaneously modify the stack pointer and general-purpose registers.

➢ Memory Access Control

For memory instructions, the Control Unit generates:

- **DMwrite** to enable memory writes
- **SAdd** to select the memory address source
- **SData** to select the data written to memory

The Control Unit supports:

- Load instructions (LDM, LDD, LDI)
- Store instructions (STD, STI)
- Stack-based memory operations (PUSH, POP, CALL, RET, RTI)

A dedicated signal (**IsLoadInstruction**) is generated to assist hazard detection and forwarding logic.

➢ Program Counter Control

The Control Unit produces a base **PCSrc** signal that defines the next PC behavior (PC+1, PC+2, jump target, stack return, or interrupt vector).

For control-flow instructions:

- Branches use static prediction (PC+1 by default)
- Jumps, calls, returns, and loops rely on the **Jump Unit** to override PC selection dynamically
- Interrupts have the highest priority and force the PC to the interrupt vector

➢ Condition Code Register (CCR) Control

The Control Unit determines when condition flags should be updated:

- **CCR_WE** enables flag updates for arithmetic, logic, and loop instructions
- For RTI (Return from Interrupt), the Control Unit activates **restore_int** to restore saved flags

Flag-only instructions such as SETC and CLRC update the CCR without modifying registers or memory.

➢ Stack Pointer Management

The Control Unit explicitly manages stack-related operations:

- **spWriteEnable** controls stack pointer updates
- PUSH and CALL decrement the stack pointer
- POP, RET, and RTI increment the stack pointer

Stack memory accesses are coordinated with ALU operations and memory addressing logic.

➢ Interrupt Handling

Interrupts override all normal instruction behavior:

- The PC is redirected to the interrupt vector
- Flags are saved externally by the CCR
- Normal pipeline flow is flushed by the hazard logic

RTI restores both the PC and the saved flags, returning execution to the interrupted context.

➢ Pipeline and Hazard Support

To support correct pipelined execution, the Control Unit provides explicit metadata for hazard management:

- **sourceRegister1 / sourceRegister2**: Identify registers read by the instruction
- **useSourceRegister1 / useSourceRegister2**: Indicate whether each source is actually used
- **destinationRegister**: Identifies the register written by the instruction
- **isTwoByteInstr**: Indicates instructions that include an immediate word

These signals are consumed by the **Forwarding Unit** and **Hazard Detection Unit** to resolve data hazards and insert stalls or forwarding paths when necessary.

➢ Design Advantages

- Fully combinational decoding logic
- Clear separation between datapath control and hazard metadata
- Robust support for stack, control flow, and interrupts
- Designed for extensibility and pipeline correctness

3.6 Jump Unit

The Jump Unit is a dedicated control block responsible for handling all control-flow–changing instructions in the processor. Its primary function is to determine whether a jump should occur and which source should be used to update the Program Counter (PC).

By separating jump decision logic from the main Control Unit, the design improves clarity, reduces complexity, and allows jump-related hazards to be handled more efficiently in a pipelined architecture.

➢ Inputs

The Jump Unit receives the following inputs:

- **Instruction information (from D/EX stage)**
    - op_code_DEX: instruction opcode
    - ra_DEX: sub-operation selector (used for conditional jumps)
- **Processor status flags**
    - ZF: Zero Flag from CCR
    - NF: Negative Flag
    - VF: Overflow Flag
    - CF: Carry Flag
    - ZF_ALU: Zero flag directly from the ALU (used for LOOP instruction)

These inputs allow the Jump Unit to evaluate both **conditional and unconditional jump instructions** accurately.

➢ Outputs

The Jump Unit produces two control signals:

- jump_now
    - Indicates whether a jump should be taken in the current cycle
    - When asserted, it overrides the normal PC update logie
- jump_target_Scr
    - Selects the source of the jump target
    - Values are aligned with the PC source encoding:
        - 3 → jump target from register R[rb]
        - 4 → jump target from memory (used for RET and RTI)

These signals are forwarded to the PC module, where the final PC selection is performed.

➢ Conditional Jump Instructions

For conditional branch instructions (JZ, JN, JC, JV), the Jump Unit:

- Examines the relevant status flag
- Asserts jump_now only if the condition is satisfied
- Selects R[rb] as the jump target

Supported conditions:

- **JZ**: Jump if Zero Flag is set
- **JN**: Jump if Negative Flag is set
- **JC**: Jump if Carry Flag is set
- **JV**: Jump if Overflow Flag is set

This allows conditional branching to be evaluated independently of the Control Unit.

➢ Loop Instruction Handling

The **LOOP** instruction is treated as a special case:

- The ALU decrements the loop counter
- The Jump Unit evaluates ZF_ALU
- If the result is **not zero**, the loop continues and a jump is taken
- If the result is zero, execution proceeds sequentially

This design ensures correct loop termination while avoiding dependence on the CCR update timing.

➢ Unconditional Control Flow Instructions

The Jump Unit also handles unconditional control-flow instructions:

- **JUMP** and **CALL**
  - Always assert jump_now
  - Jump target is taken from R[rb]
- **RET** and **RTI**
  - Always assert jump_now
  - Jump target is taken from memory (typically the stack)

This guarantees immediate redirection of program flow for subroutine calls and returns.

➢ Default Behavior

For all other instructions:

- jump_now is deasserted
- The PC continues normal sequential execution

This ensures safe operation and avoids unintended jumps.

➢ Design Advantages

The Jump Unit design provides:

- Clear separation of control-flow logic from instruction decoding
- Centralized handling of conditional and unconditional jumps
- Reduced complexity in the main Control Unit
- Improved pipeline control and hazard handling
- Scalability for future branch or jump instructions

3.7 Forwarding Unit

In a pipelined processor, data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed its write-back stage. Without special handling, such dependencies would require pipeline stalls, reducing performance.

The **Forwarding Unit** is designed to **resolve data hazards dynamically** by forwarding the most recent data values directly from later pipeline stages to earlier ones, avoiding unnecessary stalls whenever possible.

This unit primarily handles:

- **ALU-to-ALU data dependencies**
- **Load-use forwarding from the MEM/WB stage**
- **Store instruction dependencies**, including store-after-load and store-after-ALU case

➢ Inputs Overview

The Forwarding Unit monitors multiple pipeline stages:

- **ID/EX stage**
  - Source register indices used by the current instruction
  - Control signals indicating whether each source register is actually used
- **EX/MEM stage**
  - Destination register of the instruction ahead in the pipeline
  - Write-enable signal indicating whether the instruction produces a result
- **MEM/WB stage**
  - Destination register and write-enable signal
  - Instruction type information (specifically whether the instruction is a LOAD)
- **Store instruction metadata**
  - Indicates whether the current instruction is a STORE
  - Identifies which registers provide the store address and store data

➢ ALU Operand Forwarding

The unit generates two control signals:

- aluInputASelect
- aluInputBSelect

These signals control multiplexers that select the correct operands for the ALU.

Priority Rules

1. **EX/MEM Stage Has Highest Priority**
    - If the EX/MEM instruction writes to a register that matches one of the ID/EX source registers, its ALU result is forwarded directly.
    - This resolves **back-to-back ALU dependencies.**
2. **MEM/WB Stage as Secondary Source**
    - If no EX/MEM match exists, the MEM/WB stage is checked.
    - If the instruction in MEM/WB is:
        - A **LOAD**, the forwarded value comes from memory output.
        - Otherwise, the forwarded value comes from the ALU result.
3. **Default Case**
    - If no hazard is detected, operands are taken directly from the register file.

This priority-based approach ensures that the **most recent and correct value** is always used.

➢ Store Instruction Forwarding

Store instructions present a special challenge because:

- They do not write to a register
- They **consume two operands:**
    - One for the **memory address**
    - One for the **data to be written**

The Forwarding Unit therefore includes dedicated logic for store-related hazards.

Store Address Forwarding

- If the store's address register matches the destination register of a preceding instruction in the MEM/WB stage, the updated value is forwarded instead of using the stale register file value

Store Data Forwarding

- If the store's data register matches the MEM/WB destination register:
    - The forwarded value comes from:
        - **Memory output** if the previous instruction is a LOAD
        - **ALU result** otherwise

This logic correctly resolves:

- **LOAD → STORE hazards**
- **ALU → STORE hazards**

➢ Design Characteristics

- Fully **combinational logic**, ensuring no added latency
- Uses **explicit register usage signals** to avoid unnecessary comparisons
- Applies **clear prioritization** between EX/MEM and MEM/WB stages
- Extends forwarding support beyond ALU operations to include memory operations

3.8 Hazard Detection Unit

In a pipelined processor, not all hazards can be resolved through forwarding. Some situations require **stalling or flushing pipeline stages** to preserve correct program execution. The **Hazard Detection Unit** is responsible for identifying such cases and generating the necessary control signals to:

- Stall the pipeline when data is not yet available
- Insert bubbles (NOPs) when required
- Flush incorrect instructions following control-flow changes

This unit focuses on handling:

- **Load-use data hazards**
- **Control hazards caused by taken branches or jumps**
- **Pipeline synchronization for multi-byte instructions**

➢ General Operation

The Hazard Detection Unit operates as a **purely combinational block**. It continuously monitors:

- Instruction dependencies between the **ID**, **EX**, and **MEM** stages
- Control-flow decisions (branch or jump taken)
- Special conditions such as multi-byte instruction handling

Based on this information, it controls:

- Program Counter updates
- IF/ID and ID/EX pipeline register behavior

➢ Control Hazard Handling (Highest Priority)

When a **branch or jump is taken**, instructions that have already been fetched or decoded along the wrong path must be discarded.

In this case, the unit:

- Flushes the **IF/ID** pipeline register
- Flushes the **ID/EX** pipeline register
- Allows the Program Counter to continue updating normally

This ensures that no incorrect instructions proceed further in the pipeline after a control-flow change.

➢ Load-Use Data Hazard Detection

A **load-use hazard** occurs when:

- An instruction in the **EX stage** is a LOAD
- The following instruction in the **ID stage** requires the loaded value as one of its source operands

Since the data from memory is not yet available, forwarding alone cannot resolve this dependency.

When such a hazard is detected, the unit:

- **Disables the Program Counter update** (PC stall)
- **Disables the IF/ID pipeline register** (prevents fetching a new instruction)
- **Flushes the ID/EX pipeline register**, inserting a bubble

This introduces a one-cycle stall, allowing the load instruction to complete before the dependent instruction executes.

➢ Register Usage Awareness

To avoid unnecessary stalls, the hazard unit checks:

- Whether each source register in the ID stage is actually used
- Whether the destination register of the load instruction matches any used source register

This selective comparison improves efficiency and avoids false hazard detection.

➢ Multi-Byte Instruction Handling

For instructions that span multiple bytes (such as immediate-based instructions), the pipeline must wait until all required instruction bytes are available.

When a decode-stage flush request is asserted:

- The **ID/EX pipeline register is flushed**, inserting a bubble
- The rest of the pipeline continues operating normally

This mechanism ensures correct synchronization between instruction fetch and decode without stalling the entire pipeline.

➢ Design Characteristics

- Fully **combinational logic**
- Clear **priority ordering**:
    1. Control hazards
    2. Load-use hazards
    3. Multi-byte instruction handling
- Minimal performance impact by stalling only when strictly necessary

3.9 Von Neumann Memory Unit

The Von Neumann Memory Unit implements a unified memory architecture where **both instructions and data share the same physical memory array**. This design choice simplifies hardware complexity and matches the architectural requirements of the processor, while still supporting pipelined execution through careful control of access paths.

The memory unit interacts with multiple pipeline stages simultaneously, supporting instruction fetch, data read, and data write operations in a coordinated manner.

➢ Unified Instruction and Data Storage

The memory consists of a single 256-byte array:

- The **instruction fetch** always uses the Program Counter (PC) as the address.
- **Data memory access** uses a selectable address generated from the execute or memory stages.

To avoid conflicts:

- Instruction reads are purely combinational.
- Data writes occur only on the rising edge of the clock.

➢ Instruction Fetch Path

The instruction output (inst_Out) is continuously read from memory using the current PC value.
This allows the instruction fetch stage to retrieve instructions without stalling data memory operations.

This design enables:

- Continuous instruction flow
- Clean separation between fetch and memory-write timing

➢ Data Memory Address Selection

The effective data memory address is selected through a flexible multiplexer controlled by SAdd.
Possible address sources include:

- PC + 1 (used for stack and call operations)
- ALU result (used for computed addresses)
- Register file outputs (R[ra], R[rb])
- Immediate values (used in direct addressing modes)

This allows the same memory to support:

- Load and store instructions
- Stack operations (PUSH, POP, CALL, RET, RTI)
- Immediate-based addressing modes

➢ Forwarding Support for Store Instructions

To correctly support pipelined execution, the memory unit integrates **store forwarding logic**, allowing it to receive the most recent data even if it has not yet been written back to the register file.

Two dedicated control signals enable this:

- **storeAddressSelect**: Chooses between the normal address path or forwarded address values
- **storeDataSelect**: Chooses between the normal data source or forwarded ALU/memory results

This mechanism ensures correctness for:

- STORE after ALU instructions
- STORE after LOAD instructions
- Stack operations dependent on recently updated values

➢ Data Write Path

The data written to memory is selected using SData, which chooses between:

- Register file output (typical STORE)
- PC + 1 (used during CALL instructions to save the return address)

The final write data may also be overridden by forwarded values when required, ensuring correct behavior in all pipeline scenarios.

All memory writes occur synchronously on the rising clock edge when DMwrite is asserted.

➢ Data Read Path

Data memory reads are purely combinational.
The selected memory location is read immediately and provided as Data_Memory_Out, allowing:

- LOAD instructions to retrieve data efficiently
- POP, RET, and RTI instructions to access stack values

➢ Reset and Interrupt Handling

On reset:

- Only the **data memory region** is cleared (upper half of memory)
- Instruction memory remains intact

This allows programs to persist while ensuring a clean data state.

Interrupt handling logic reserves space for interrupt vectors and integrates seamlessly with the Program Counter and CCR mechanisms.

3.10 Pipeline Registers

Pipeline registers are responsible for **separating pipeline stages and preserving intermediate results and control signals** between clock cycles. Each pipeline register captures the outputs of one stage and forwards them to the next stage, enabling multiple instructions to be processed concurrently.

In this processor, pipeline registers are enhanced with:

- **Enable signals** to support stalls
- **Flush signals** to handle control hazards, interrupts, and incorrect speculative execution
- Dedicated fields for **forwarding and hazard detection**
- Explicit support for **two-byte instructions with immediate operands**

The pipeline consists of four main pipeline registers:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

➢ IF/ID Pipeline Register

The IF/ID register separates the **Instruction Fetch (IF)** and **Instruction Decode (ID)** stages. Its primary function is to store the fetched instruction and the corresponding PC + 1 value.

A key feature of this register is its **support for two-byte instructions**. When an instruction that requires an immediate operand is detected:

- The pipeline temporarily enters a waiting state
- The second byte (immediate value) is captured separately
- A controlled flush is generated to prevent premature execution

This mechanism ensures that immediate values are always correctly aligned with their instructions without breaking pipeline timing.

The IF/ID register also supports:

- **Stalling** through the enable signal (used during load-use hazards)
- **Flushing** during branch mispredictions, jumps, interrupts, or control hazards

➢ ID/EX Pipeline Register

The ID/EX register transfers decoded instruction information from the **Decode (ID)** stage to the **Execute (EX)** stage.

It carries:

- Instruction fields (opcode, register indices)
- Register file outputs
- Immediate values and immediate-valid flags
- ALU control signals
- Memory access control signals
- Register write-back control signals
- Condition Code Register (CCR) write enable
- Stack control signals

Additionally, this register propagates **hazard detection and forwarding metadata**, including:

- Source register identifiers
- Destination register identifier
- Flags indicating whether source registers are actually used

Flushing this register inserts a **bubble** into the pipeline, which is essential for resolving load-use hazards and control hazards safely.

➢ EX/MEM Pipeline Register

The EX/MEM register isolates the **Execute (EX)** stage from the **Memory (MEM)** stage.

It stores:

- ALU results
- Data intended for memory access
- Immediate values needed for memory addressing
- Register file values required for store operations
- Control signals for memory writes and register writes
- Stack pointer update signals

This register plays a crucial role in:

- Supporting **store-data and store-address forwarding**
- Passing load instruction indicators to later stages
- Providing forwarding sources for subsequent instructions

Flushing EX/MEM is typically required after taken branches, jumps, or incorrect speculative execution.

➢ MEM/WB Pipeline Register

The MEM/WB register connects the **Memory (MEM)** stage to the **Write-Back (WB)** stage.

It holds:

- ALU results
- Data read from memory
- Destination register information
- Register write enable signals
- Stack pointer write enable signals
- Load instruction indicators

This register determines the **final value written back to the register file**, selecting between:

- ALU output
- Data memory output

It also provides the final forwarding sources for instructions in earlier pipeline stages.

➢ Pipeline Control Features

Across all pipeline registers, the following control mechanisms are consistently applied:

- **Enable signals** to freeze pipeline stages during stalls
- **Flush signals** to invalidate incorrect or unsafe instructions
- **Immediate-handling logic** to support variable-length instructions
- **Forwarding metadata propagation** for data hazard resolution

➢ Design Benefits

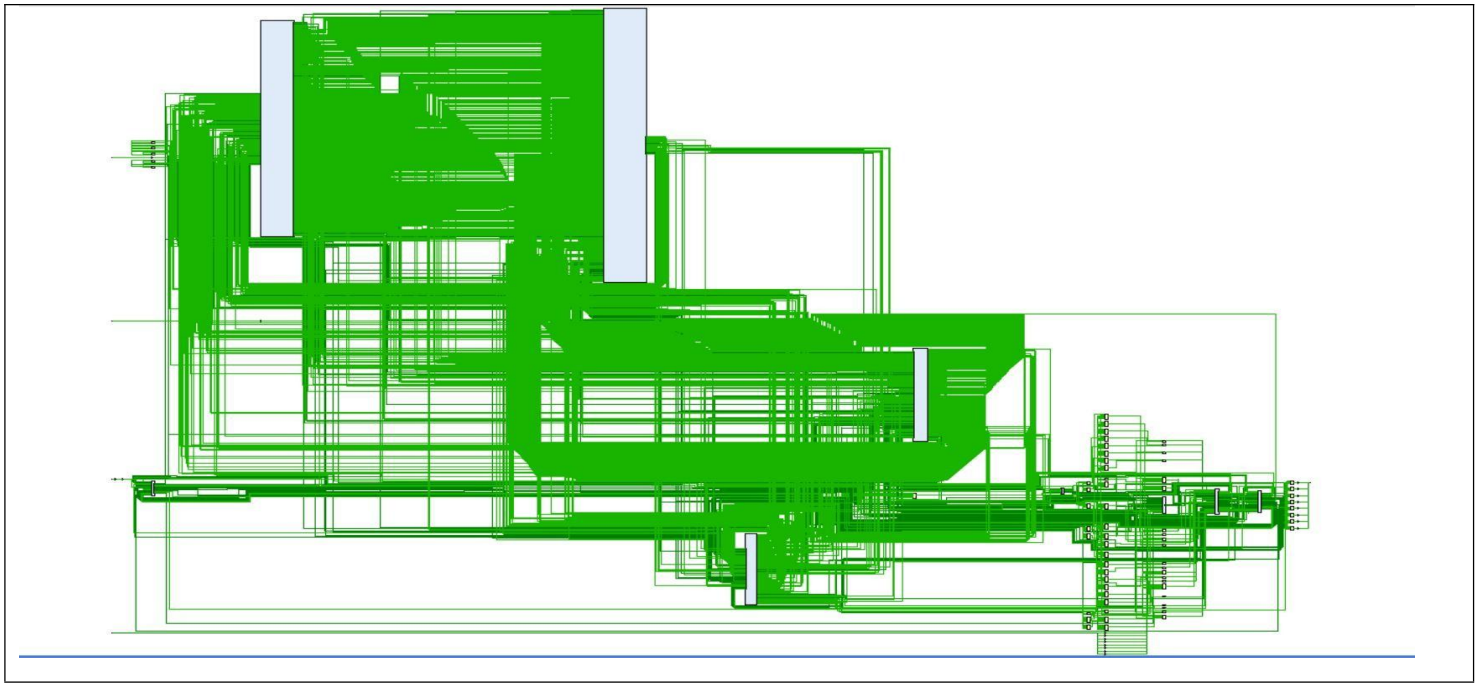
The pipeline register design ensures:

- Correct execution of variable-length instructions
- Robust handling of data and control hazards
- Clean separation between pipeline stages
- Efficient forwarding and minimal pipeline stalls
- Full compatibility with interrupts, calls, and returns

4. Synthesis And Implementation

4.1 Synthesis of the Processor Design

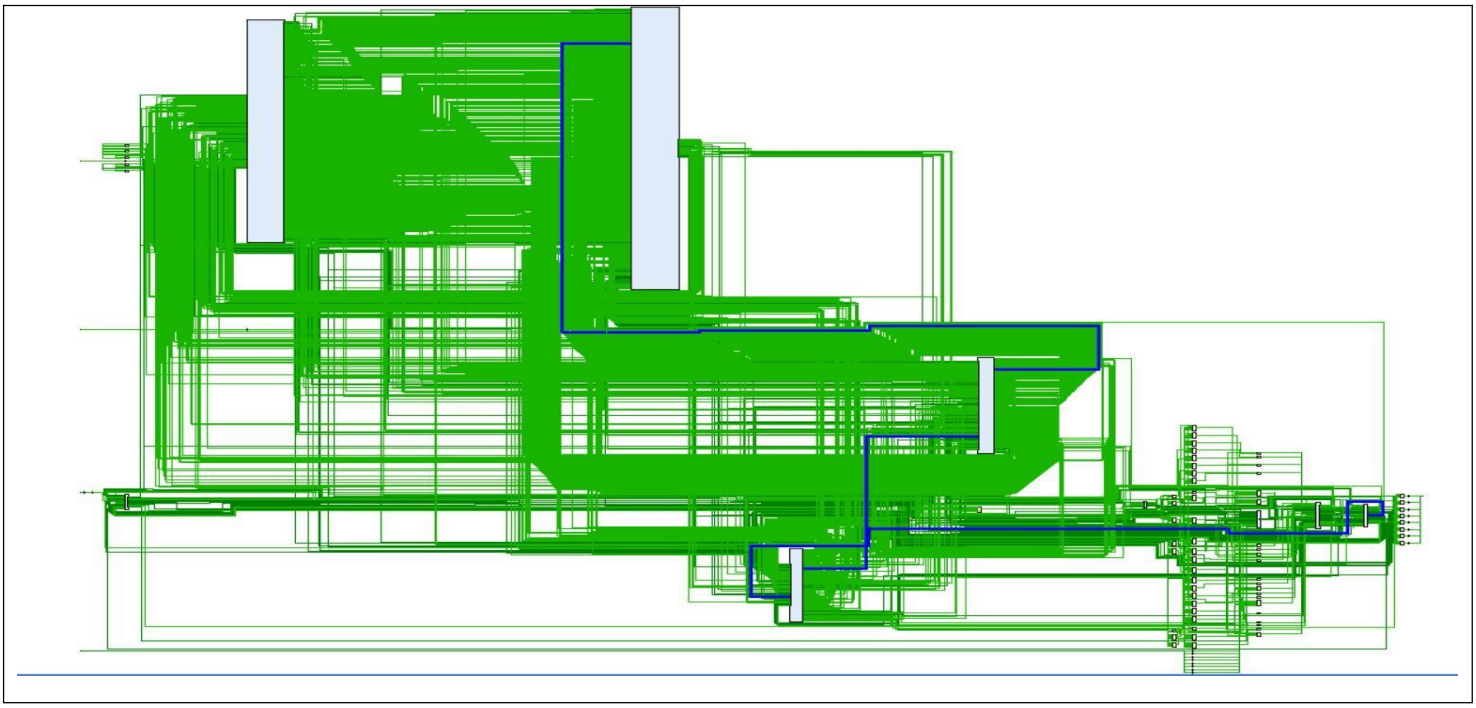
The synthesis of the proposed pipelined processor was performed targeting a **Xilinx Kintex-7 FPGA**, using the **xc7k70tfbv676-1** device, which represents the selected product family and development platform for this project. All RTL modules were synthesized under a **clock period constraint of 10 ns**, corresponding to an operating frequency of **100 MHz**, to ensure reliable high-speed operation. The design was verified to be fully **synthesizable**, meaning that all Verilog constructs used in the implementation are supported by FPGA synthesis tools and can be mapped to real hardware resources. In particular, the design contains **no unintended latches**, as all combinational logic is fully specified and all storage elements are implemented using edge-triggered flip-flops. Additionally, the synthesis process completed **without critical warnings or errors**, confirming the absence of unsynthesizable constructs such as incomplete sensitivity lists, multiple drivers, or timing-dependent behavioral code. Successful synthesis on the Kintex-7 device validates that the processor architecture is hardware-realizable and suitable for FPGA implementation under the specified timing constraints.



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	2.866 ns	Worst Hold Slack (WHS):	0.066 ns	Worst Pulse Width Slack (WPWS):	4.650 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	2766	Total Number of Endpoints:	2766	Total Number of Endpoints:	2321

All user specified timing constraints are met.



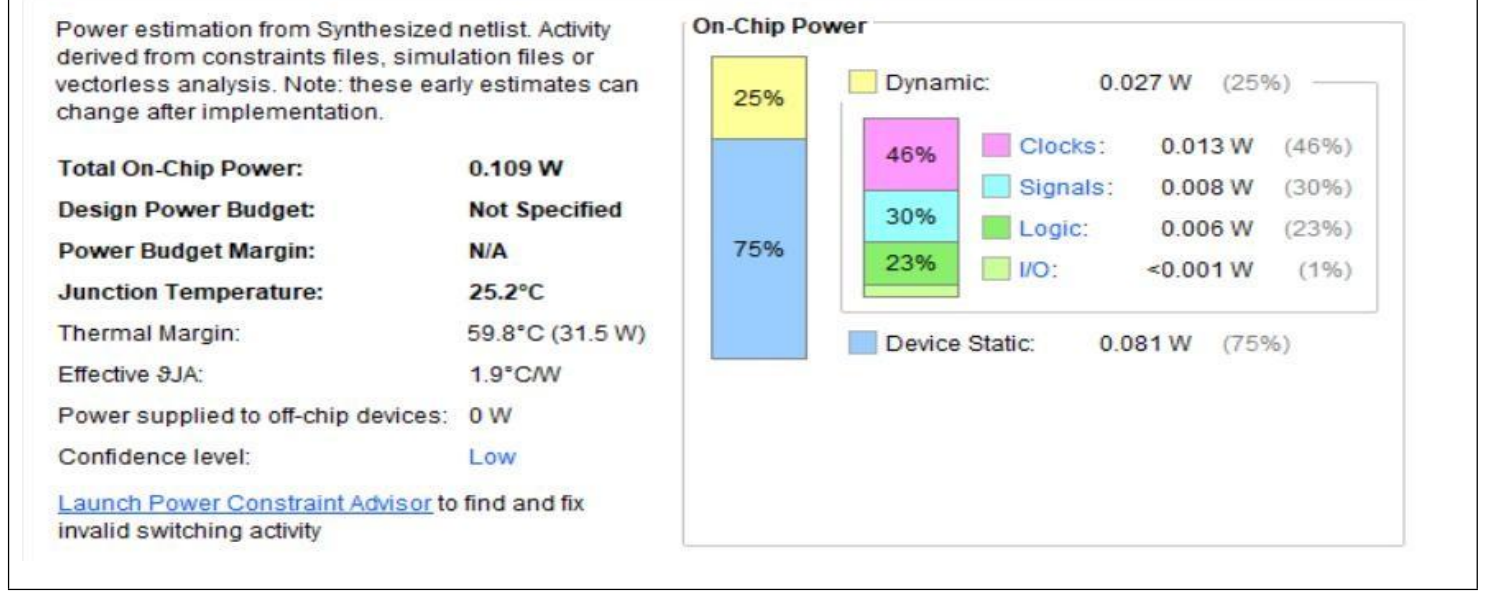
Name	Slice LUTs (41000)	Slice Registers (82000)	F7 Muxes (20500)	F8 Muxes (10250)	Bonded IOB (300)	BUFGCTRL (32)
Top_Module	1809	2320	550	272	19	1
ALU_unit (ALU)	284	0	5	0	0	0
CCR_unit (CCR)	150	4	1	0	0	0
DEX (D_EX)	0	68	0	0	0	0
EXM (EX_MEM)	0	72	0	0	0	0
FD (IF_ID)	6	27	0	0	0	0
Memory (Von_Neumann_Mem)	1153	2048	544	272	0	0
MWB (MEM_WB)	0	53	0	0	0	0
PC_unit (PC)	106	8	0	0	0	0
RF (RegisterFile)	59	32	0	0	0	0

Summary

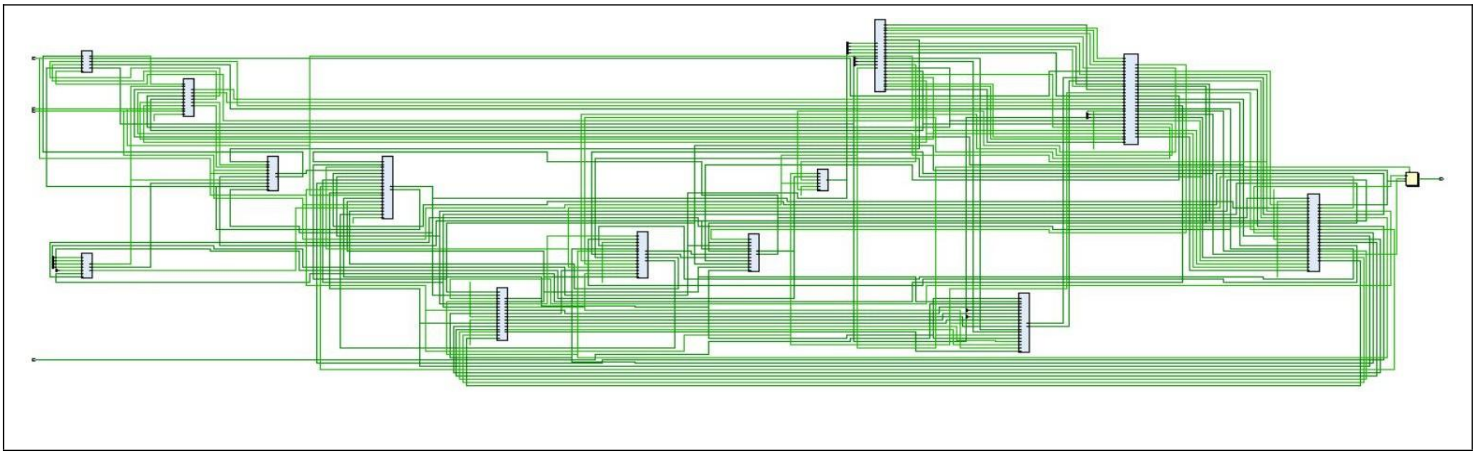
Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	**0.109 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.2°C**
Thermal Margin:	59.8°C (31.5 W)
Effective ϑJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



4.2 Elaboration



Elaborated Design (36 infos)

General Messages (36 infos)

[Synth 8-6157] synthesizing module 'Top_Module' [Top_Module.v:2] (13 more like this)

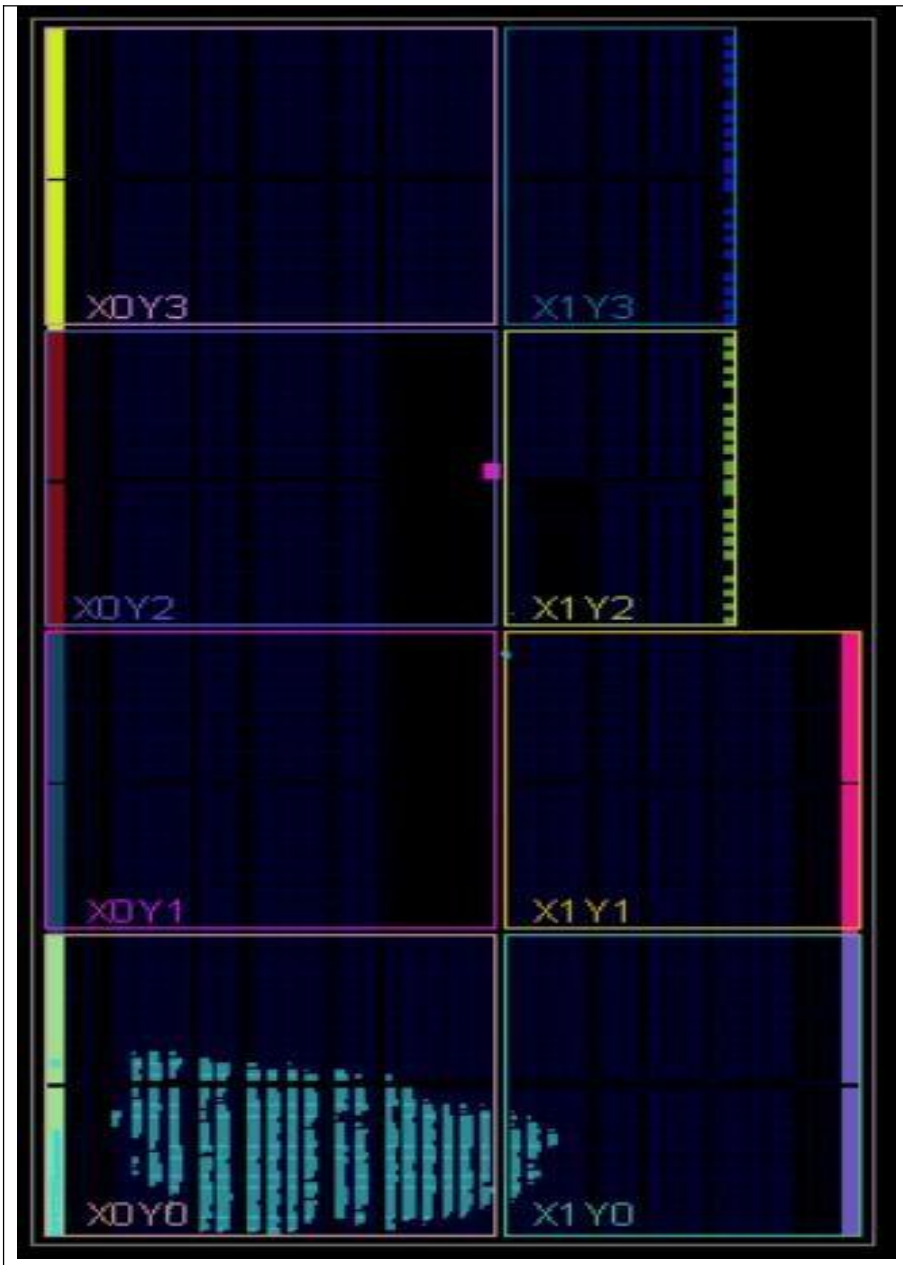
[Synth 8-6155] done synthesizing module 'PC' (1#1) [PC.v:2] (13 more like this)

[Synth 8-226] default block is never used [ALU.v:29] (1 more like this)

[Project 1-570] Preparing netlist for logic optimization

[Common 17-14] Message 'Constraints 18-402' appears 100 times and further instances of the messages will be disabled. Use the Tcl command set_msg_config to change the current settings.

4.3 Implementation



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.226 ns	Worst Hold Slack (WHS):	0.049 ns	Worst Pulse Width Slack (WPWS):	4.600 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	4541	Total Number of Endpoints:	4541	Total Number of Endpoints:	2321

All user specified timing constraints are met.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.12 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.2°C**
Thermal Margin:	59.8°C (31.5 W)
Effective ϑJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

Name	Slice LUTs (41000)	Slice Registers (82000)	F7 Muxes (20500)	F8 Muxes (10250)	Slice (10250)	LUT as Logic (41000)	LUT Flip Flop Pairs (41000)	Bonded IOB (300)	BUFGCTRL (32)
N Top_Module	1755	2320	550	272	841	1755	75	19	1
ALU_unit (ALU)	97	0	5	0	81	97	0	0	0
CCR_unit (CCR)	282	4	1	0	201	282	4	0	0
DEX (D_EX)	0	68	0	0	30	0	0	0	0
EXM (EX_MEM)	0	72	0	0	45	0	0	0	0
FD (IF_ID)	5	27	0	0	9	5	1	0	0
Memory (Von_Neumann_Mem)	1163	2048	544	272	736	1163	2	0	0
MWB (MEM_WB)	0	53	0	0	33	0	0	0	0
PC_unit (PC)	100	8	0	0	54	100	0	0	0
RF (RegisterFile)	59	32	0	0	27	59	1	0	0

5. Test Cases and Verification

This section presents a set of representative test cases used to verify the correct functionality of the designed pipelined processor. The test cases were selected to cover the main architectural components, including arithmetic operations, memory access, control flow instructions, pipeline behavior, and exceptional conditions such as interrupts.

All test cases were executed using simulation, and correctness was verified by observing register values, memory contents, program counter updates, and condition flags.

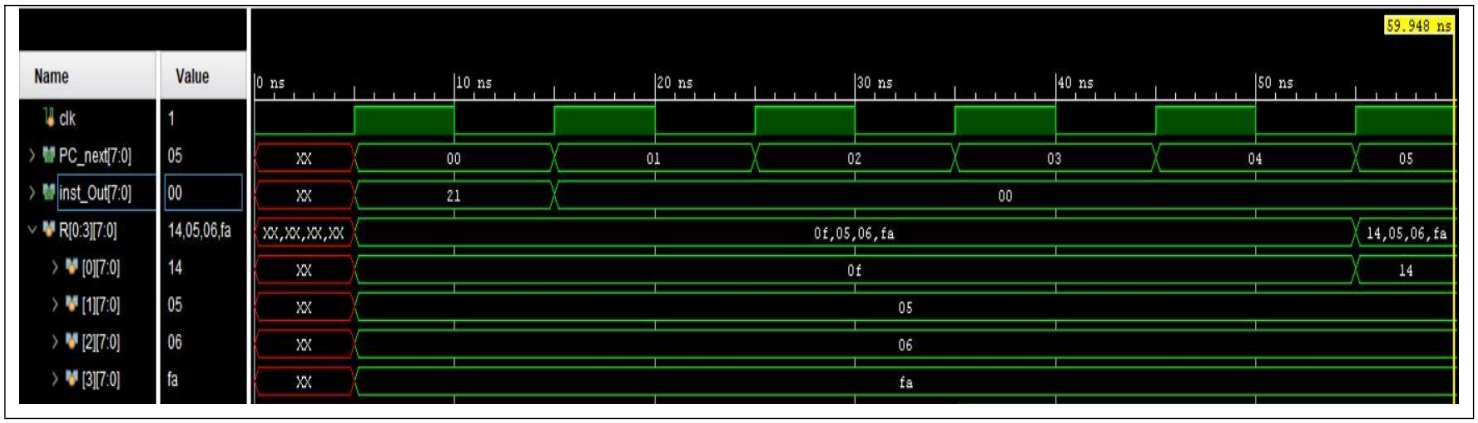
5.1 Arithmetic and Logical Instructions

These test cases validate the functionality of the **ALU** and **Condition Code Register (CCR)**.

➢ Test Case 1: ADD Instruction

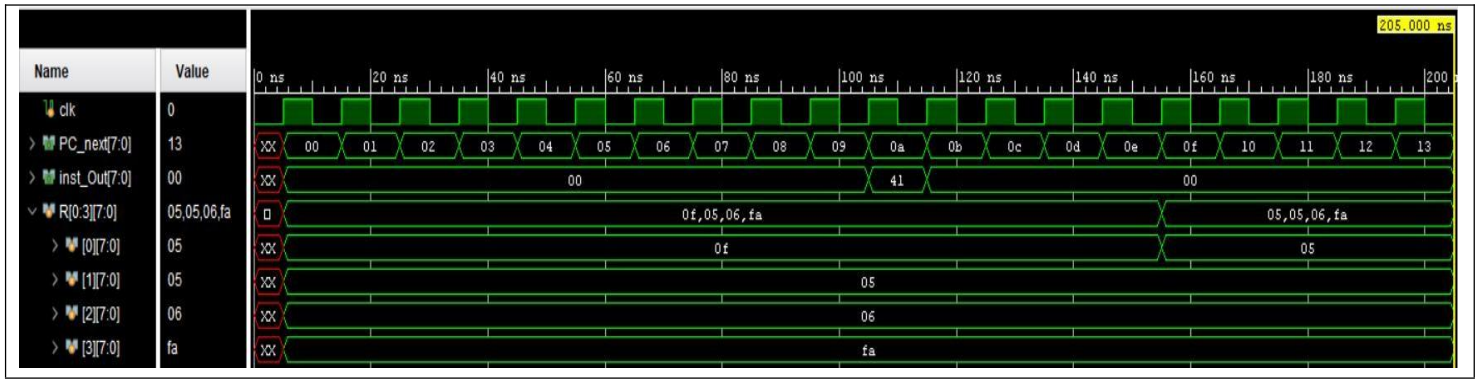
· **Operation:** ADD Ra(R[0]) Rb(R[1])

Let Ra =15 and Rb=5



➢ Test Case 2: Logical Operations AND

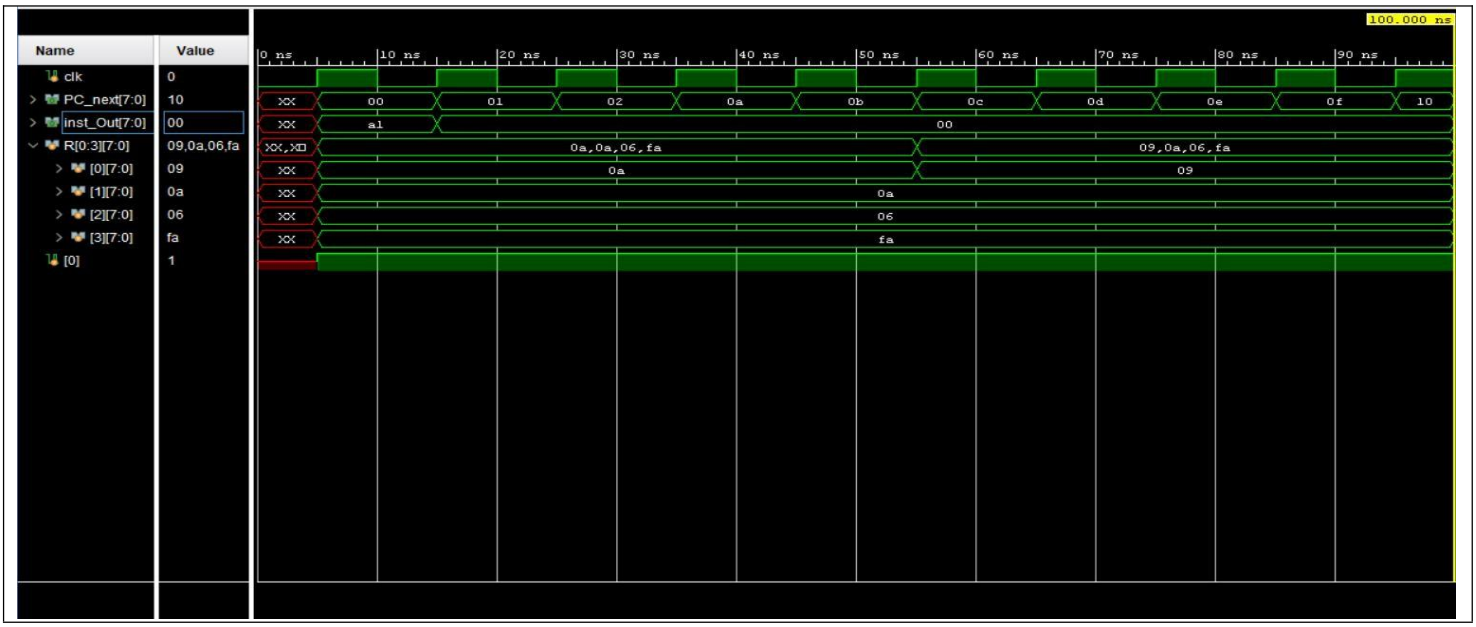
- **Operation: AND** Ra `(R[0])` Rb `(R[1])`
- Let Ra =15 and Rb=5



5.2 Loop Instructions

➢ Test Case 3: LOOP Instruction

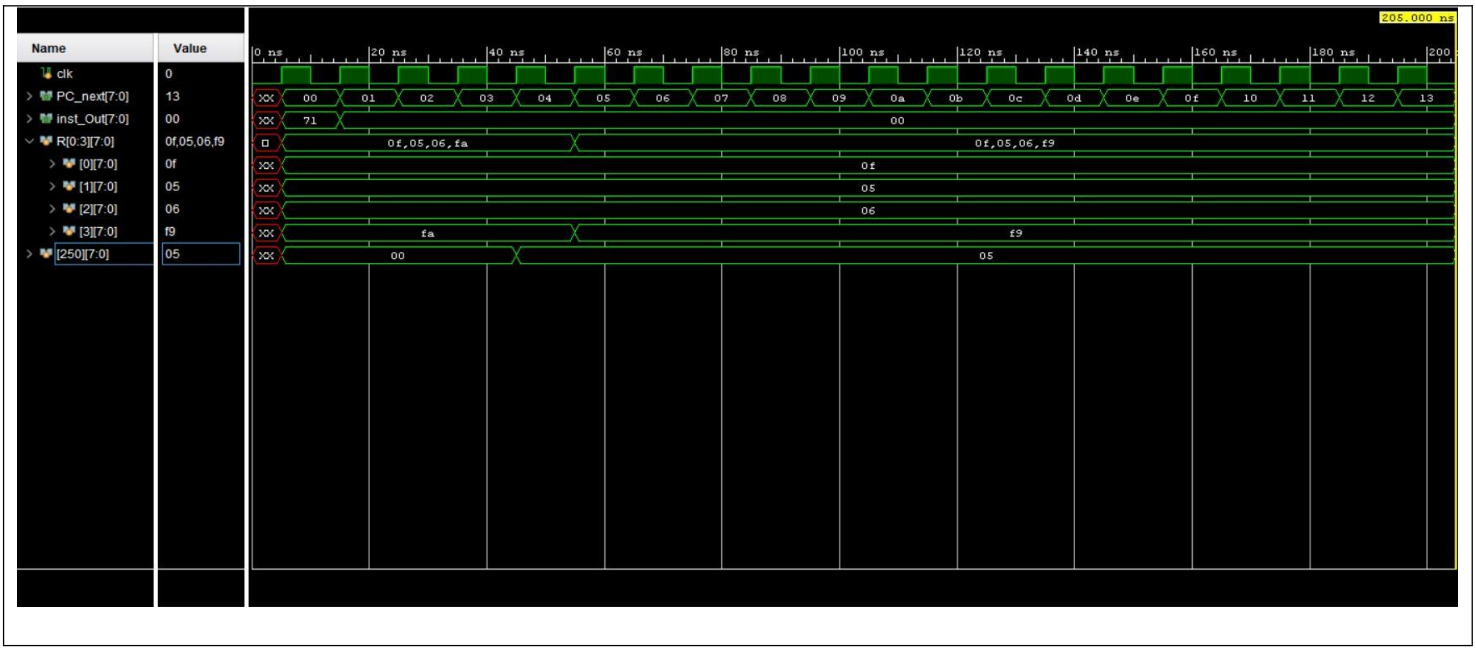
- **Operation:** LOOP



5.3 Memory Access Instructions

➢ Test Case 4: PUSH Instruction

- **Operation:** PUSH

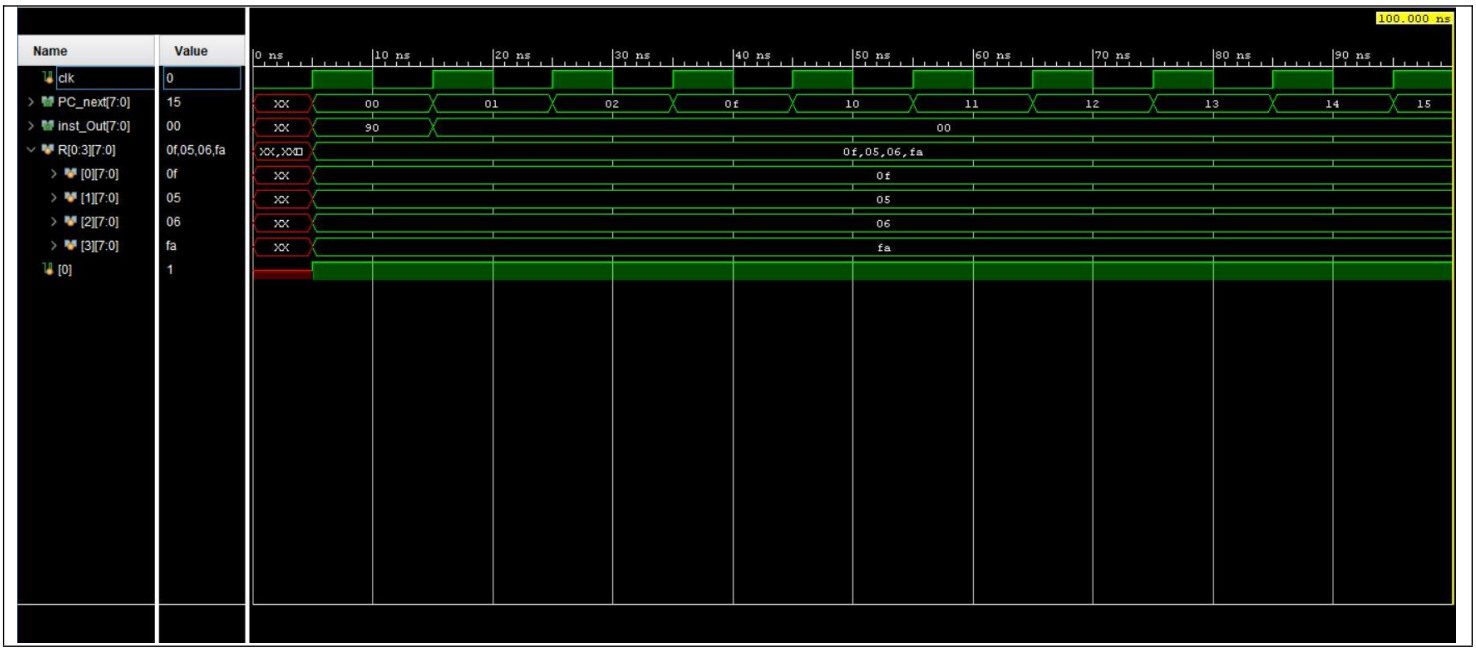


5.4 Control Flow Instructions

These test cases verify correct **Program Counter (PC)** updates and jump handling.

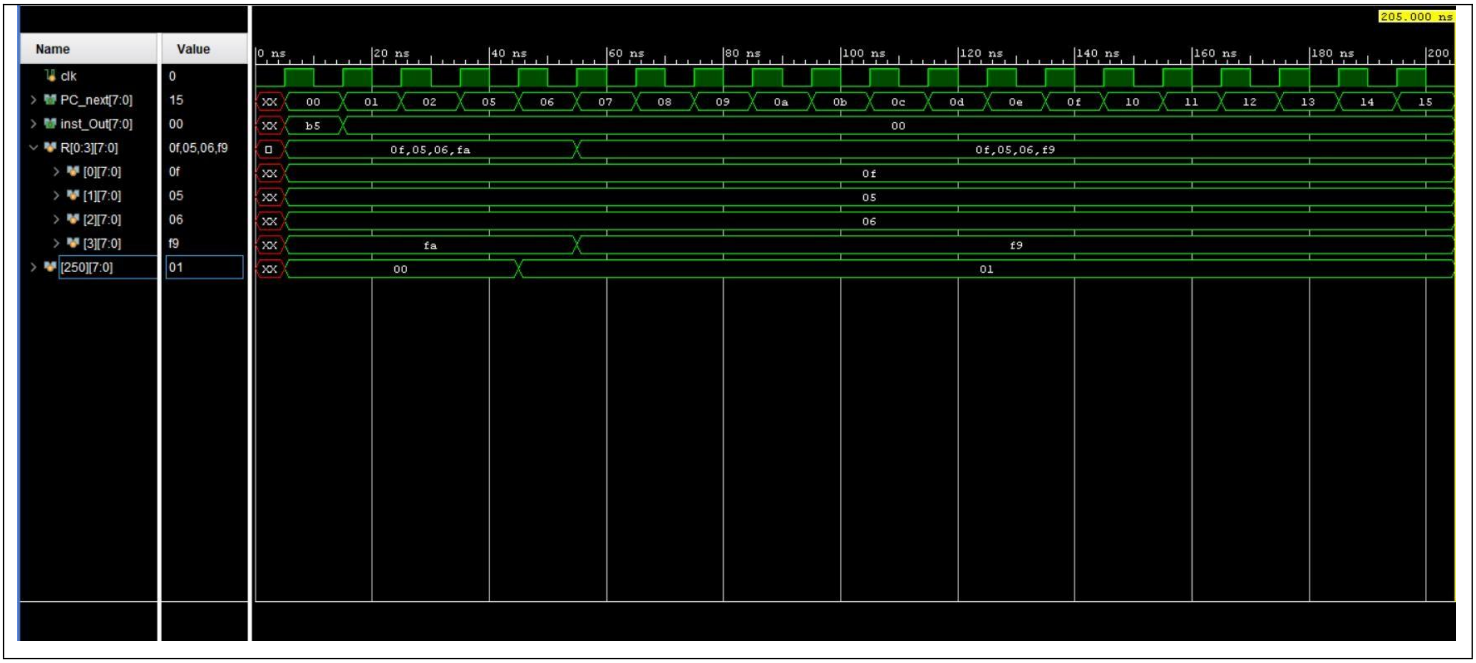
➢ Test Case 5: Conditional Jumps

- **Operations:** JZ



➢ Test Case 6: Unconditional Jumps

- **Operations:** CALL



5.5 Advanced test cases

➢ Test case 7:

ADD R[0] , R[1] F D E M W

ADD R[2] , R[0] F D E M W

ADD R[0] , R[2] F D E M W

➔ Start : R[0] = 5 , R[1] = 1 , R[2] = 0

➔ Final : R[0] = 12 , R[1] = 1 , R[2] = 6

➢ Test case 8:

ADD R[2] , R[1] F D E M W

INC R[2] F D E M W

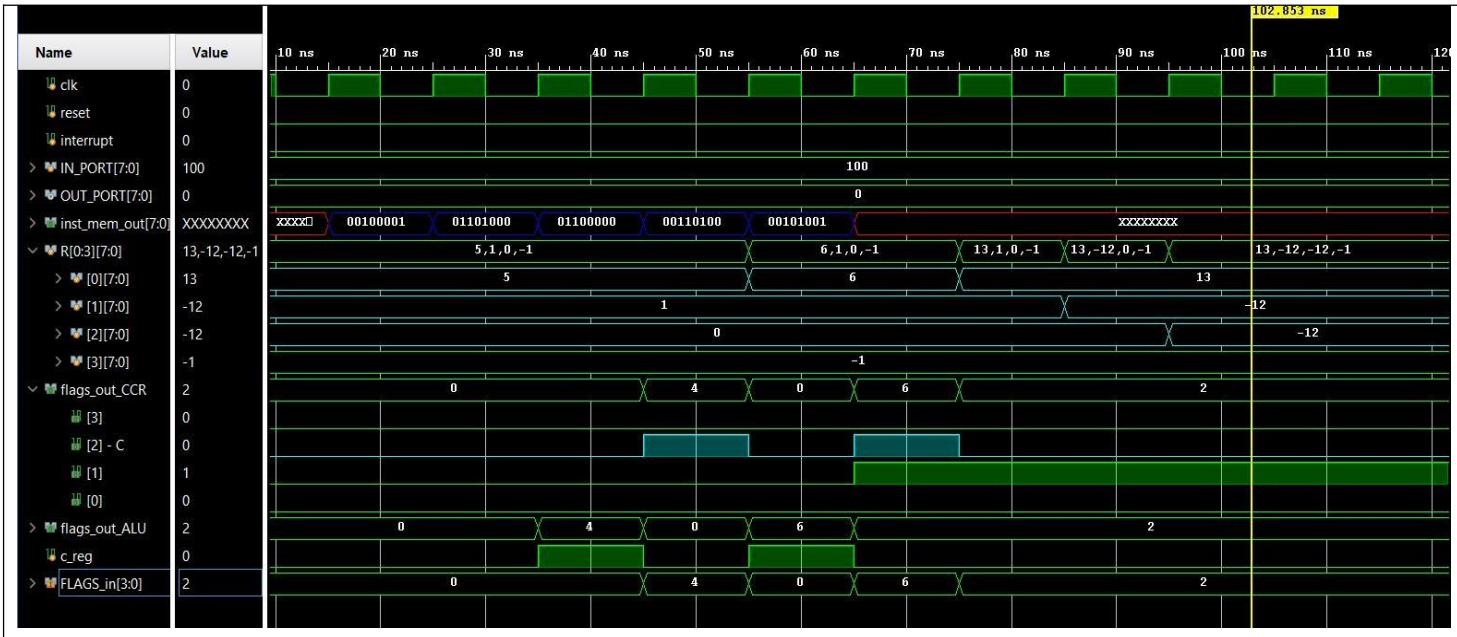
SUB R[1] , R[0] F D E M W

INC R[1] F D E M W

➔ Start : R[0] = 5 , R[1] = 3 , R[2] = 2

➔ Final : R[0] = 5 , R[1] = -3 , R[2] = 6

➢ Test case 9:



ADD	R[0] , R[1]	F	D	E	M	W				
SETC			F	D	E	M	W			
RLC	R[0]			F	D	E	M	W		
SUB	R[1] , R[0]				F	D	E	M	W	
ADD	R[2] , R[1]					F	D	E	M	W

➔ Start : R[0] = 5 , R[1] = 1 , R[2] = 0

➔ Final : R[0] = 13 , R[1] = -12 , R[2] = -12

➢ Test case 10:

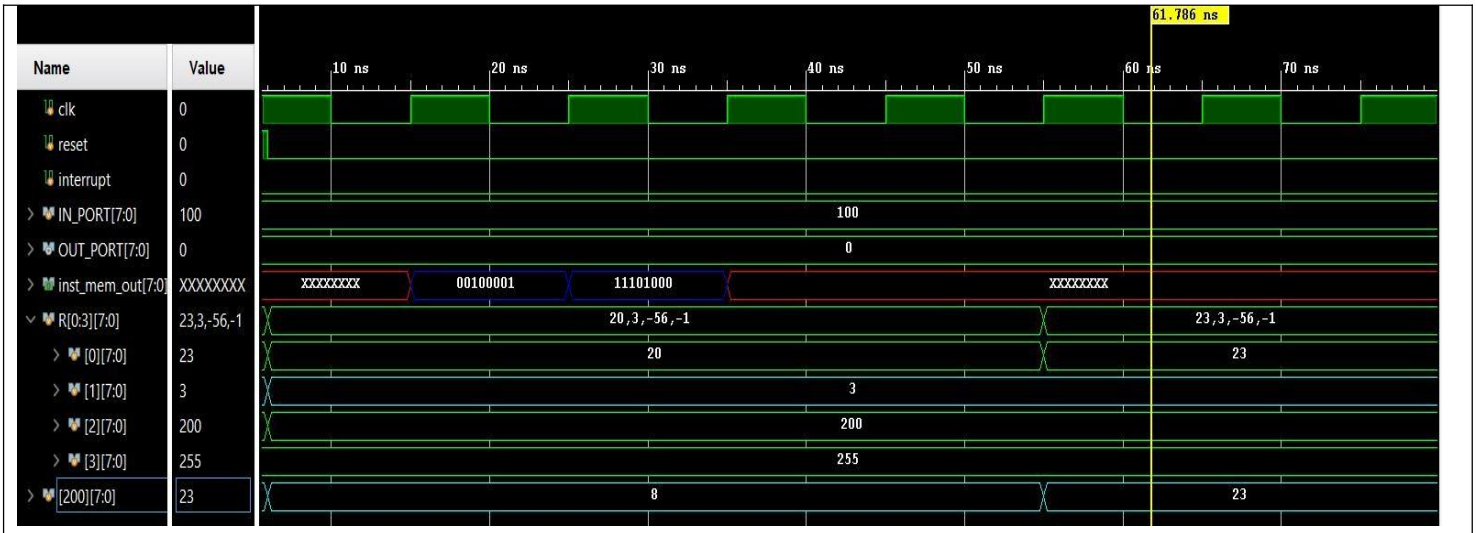
LDI R[1] ← M[R[0]] F D E M W

ADD R[1] , R[2] F D D E M W

➔ Start : R[0] = 200 , R[1] = 3 , R[2] = 2 , M[200] = 8

➔ Final : R[0] = 200 , R[1] = 10 , R[2] = 2

➢ Test caes 11:



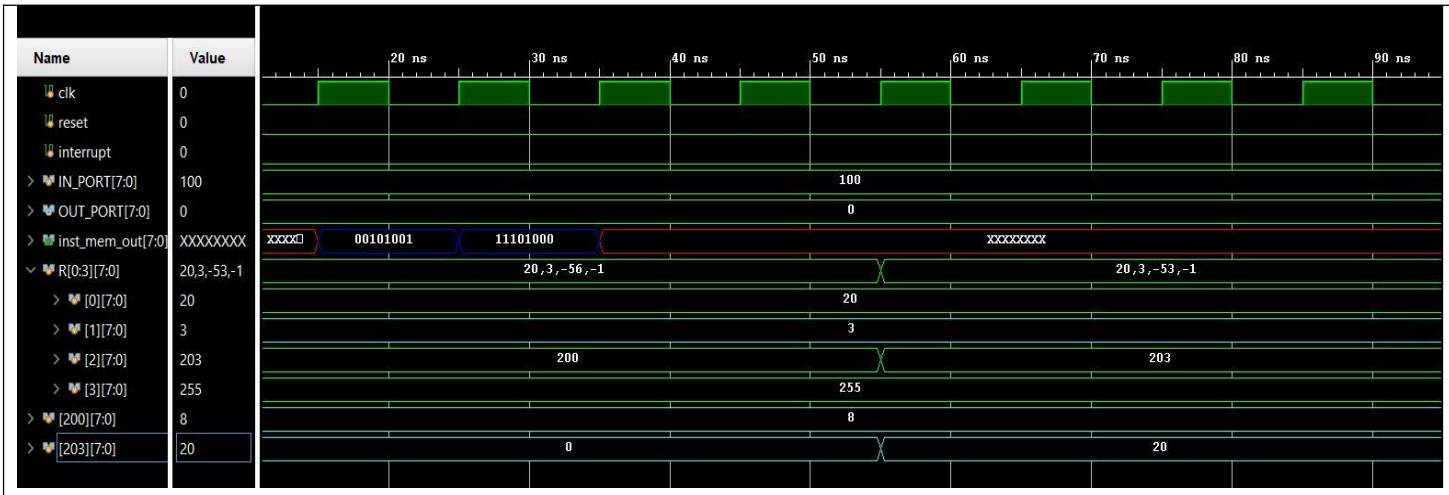
ADD R[0] , R[1] F D E M W

STI M[R[2]] ← R[0] F D E M W

➔ Start : R[0] = 20 , R[1] = 3 , R[2] = 200 , M[200] = 8

➔ Final : R[0] = 23 , R[1] = 3 , R[2] = 200 , M[200] = 23

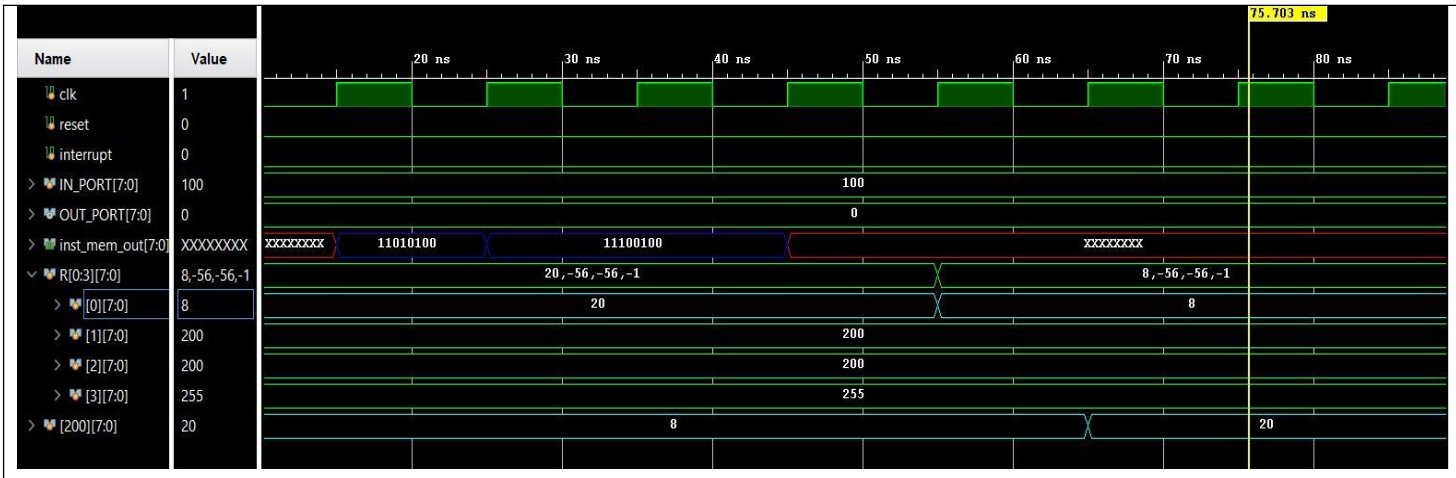
➢ Test case 12:



ADD	R[2] , R[1]	F	D	E	M	W
STI	M[R[2]] ← R[0]		F	D	E	M	W

➔ Start : R[0] = 20 , R[1] = 3 , R[2] = 200 , M[200] = 8 , M[203] = 0
➔ Final : R[0] = 20 , R[1] = 3 , R[2] = 203 , M[200] = 8, M[203] = 20

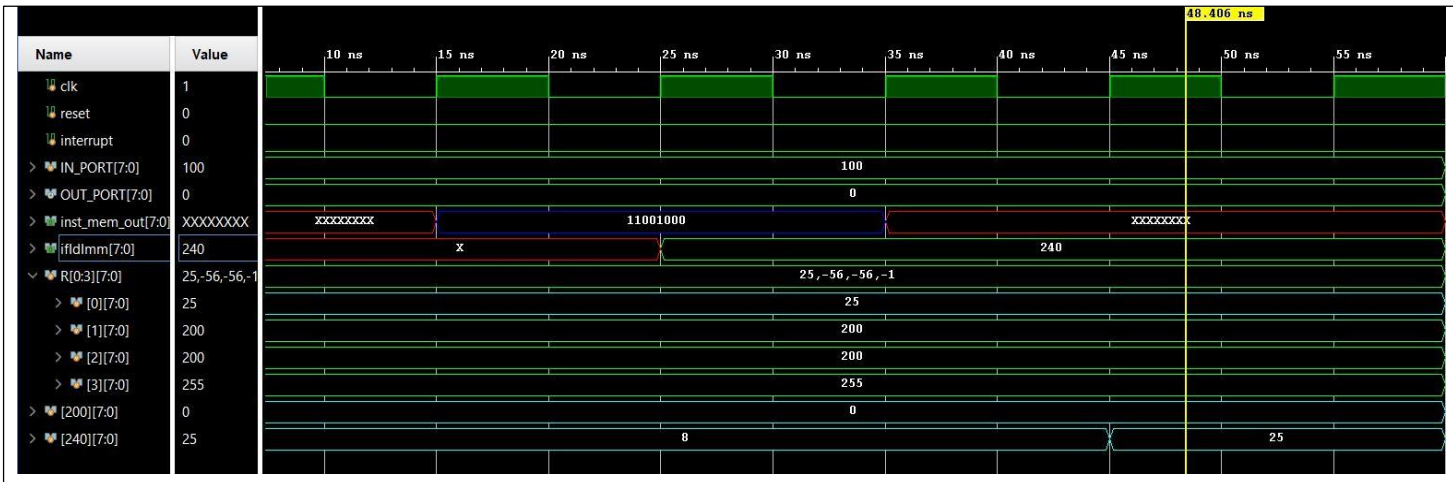
➢ Test case 13:



LDI	R[0] ← M[R[1]]	F	D	E	M	W
STI	M[R[1]] ← R[0]		F	D	E	M	W

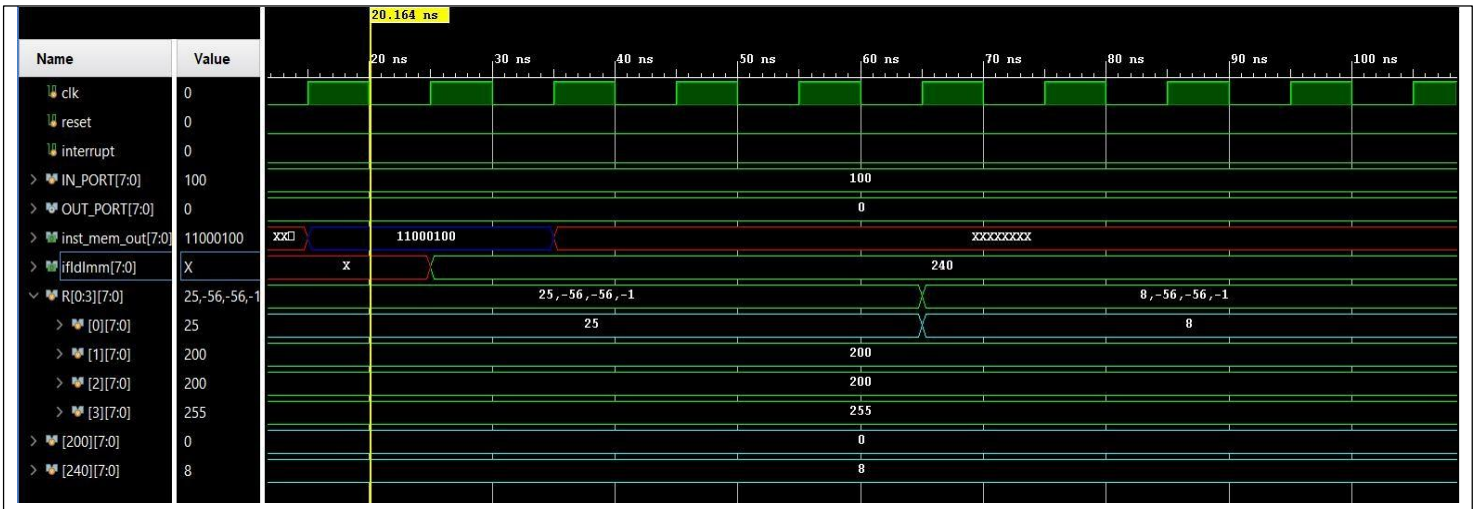
➔ Start : R[0] = 20 , R[1] = 200 , R[2] = 200 , M[200] = 8
➔ Final : R[0] = 8 , R[1] = 200 , R[2] = 200 , M[200] = 20

➢ Test case14:



M[ea] ←R[0];

➢ Test case15:



LDM R[0] ← M[imm]

6. Conclusion

In this project, a complete **pipelined processor architecture** was designed, implemented, and verified using a modular and structured approach. The processor integrates all essential components, including the Program Counter, Register File, Arithmetic Logic Unit, Condition Code Register, Control Unit, Jump Unit, memory subsystem, and pipeline registers, forming a functional multi-stage pipeline.

The design successfully supports a wide range of instructions such as arithmetic and logical operations, memory access instructions, control flow instructions, stack operations, and interrupt handling. The **Von Neumann memory architecture** was correctly implemented, allowing both instruction fetch and data access within a unified memory space while maintaining correct operation through proper control signaling.

Pipeline operation was carefully managed using **pipeline registers** between all stages, ensuring correct data and control signal propagation. Potential hazards were addressed conceptually through forwarding and hazard detection mechanisms, while control hazards were handled using flushing and program counter redirection. The Jump Unit enabled both conditional and unconditional branching based on the processor flags, ensuring accurate program flow.

Extensive simulation-based test cases confirmed the correctness of the design. The processor demonstrated proper ALU functionality, correct flag updates, reliable memory access, accurate jump behavior, and stable interrupt handling. Pipeline behavior was validated under normal execution as well as in scenarios involving hazards and control flow changes.

Overall, the implemented processor meets the intended functional requirements and demonstrates a solid understanding of pipelined processor design principles. The modular structure of the system allows for future enhancements such as deeper pipelines, cache integration, expanded instruction sets, or more advanced hazard resolution techniques.